ARCHITECTURE ORIENTÉE SERVICES · LAB 1

# Du monolithe aux services SOAP et REST

Compte rendu du projet BiblioTech
Gestion de livres avec PostgreSQL


Étudiante **Chahd Zanina**

Classe **Informatique Appliquée 3.1**

# Objectif et organisation

L'énoncé demande de faire évoluer la gestion des livres de BiblioTech en trois étapes : un script monolithique, un service SOAP avec contrat WSDL, puis une API REST. Les trois versions utilisent PostgreSQL et SQLAlchemy.

Un livre est décrit par un identifiant entier unique, un titre, un auteur, un nombre d'exemplaires disponibles et un tarif de location. Le stock et le tarif doivent être positifs ou nuls.

| Étape | Interface | Fichiers principaux | Accès aux données |
|---|---|---|---|
| Monolithe | Fenêtre PyQt5 | `monolith/library_app.py` | SQLAlchemy dans le même script |
| SOAP | Messages XML / WSDL | `soap/soap_service.py` `soap/db.py` | Couche de données séparée |
| REST | HTTP / JSON / OpenAPI | `rest/main.py` `rest/db.py` | Couche de données séparée |

### Environnement technique

Le dépôt fournit `requirements.txt` pour Python, `docker-compose.yml` pour PostgreSQL et, pour la partie REST, un `Dockerfile` et un second fichier Compose. Le service REST et le serveur SOAP écoutent tous les deux sur le port 8000 ; ils sont lancés séparément.

```
docker compose up -d db

python monolith/library_app.py

python soap/soap_service.py

docker compose -f rest/docker-compose.yml up --build
```

> **Lecture du dépôt.** Les fonctionnalités présentées ci-dessous correspondent au code et aux captures conservés dans le projet. Les captures illustrent des exécutions antérieures ; elles ne constituent pas un nouvel essai effectué pour ce rapport.

# Application monolithique

Le fichier `monolith/library_app.py` rassemble le modèle `Book`, la connexion PostgreSQL, les opérations CRUD, la validation et la fenêtre PyQt5. La table `books` comporte les cinq champs demandés. L'interface affiche la liste des livres, un formulaire de saisie et les actions Ajouter, Modifier, Supprimer et Rafraîchir.

| Opération | Implémentation observée |
| --- | --- |
| Créer | `create_book` contrôle l'ID, évite les doublons puis enregistre le livre. |
| Lire | `get_all_books` retourne les livres triés par identifiant. |
| Modifier / supprimer | `update_book` et `delete_book` recherchent le livre puis valident la transaction. |

La fonction `validate` refuse un identifiant non positif, un titre ou auteur vide, ainsi qu'un stock ou tarif négatif. Les erreurs de saisie et de base de données sont affichées par des boîtes de dialogue.

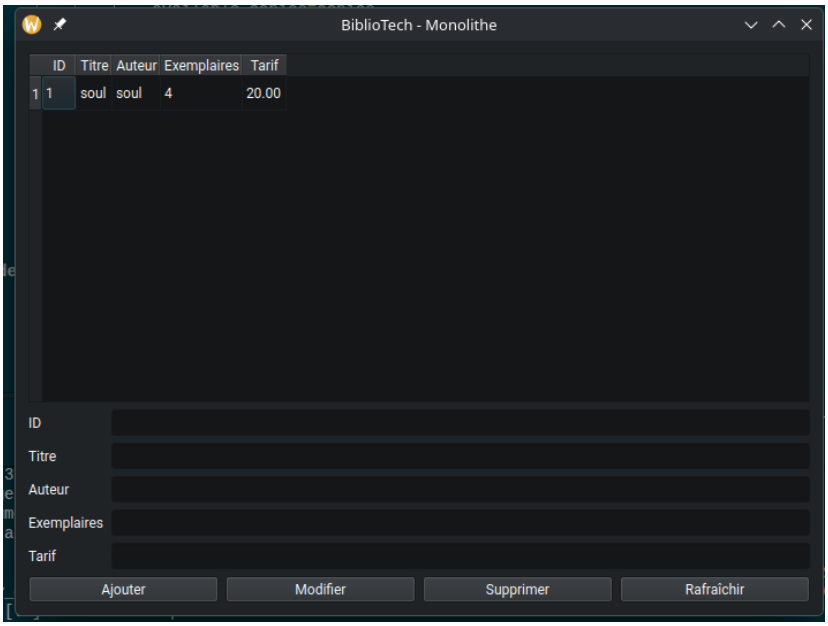



Figure 1 — Interface PyQt5 présente dans le dépôt (`monolith/assets/monolith-interface.png`).

### Limites de l'approche monolithique

Le modèle, l'accès aux données et l'interface sont réunis dans un seul script. Une modification de la logique métier peut donc affecter la fenêtre et les traitements de persistance. Cette organisation complique les tests isolés, la maintenance et la réutilisation des opérations par d'autres applications. Elle motive l'extraction de services accessibles sur le réseau dans les parties suivantes.

# Service Web SOAP

Le service `soap/soap_service.py` utilise Spyne et publie quatre opérations. Le modèle `BookModel` reprend les champs du livre. Les fonctions de persistance et de validation sont dans `soap/db.py`.

| Opération SOAP | Rôle | Fonction appelée |
|---|---|---|
| `CreateBook` | Ajouter un livre | `db.create_book` |
| `GetBook` | Consulter un livre par ID | `db.get_book` |
| `UpdateBook` | Modifier un livre | `db.update_book` |
| `DeleteBook` | Supprimer un livre | `db.delete_book` |

Le contrat est enregistré dans `soap/bibliotech.wsdl` et annoncé par le service à `http://localhost:8000/?wsdl`. La fonction `guard` transforme les erreurs de validation en `Client.Validation` et l'absence d'un livre en `Client.NotFound`.

Le fichier `soap/test_book_validation.py` définit six cas : données valides, stock négatif, tarif négatif, ID invalide, titre vide et auteur vide. Les captures Postman du dépôt illustrent les quatre opérations ; celle de création affiche une réponse HTTP 200 contenant un livre.

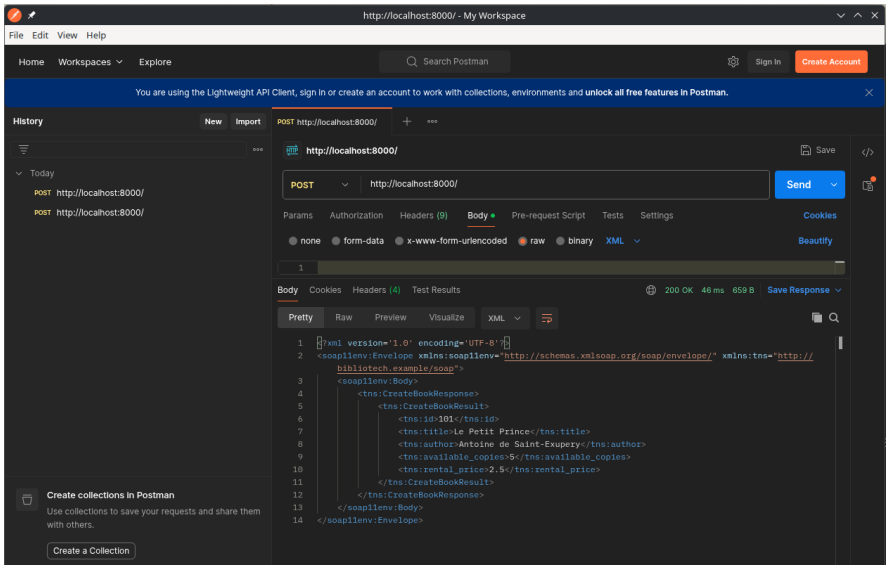



Figure 2 — Réponse de création SOAP dans Postman (`soap/assets/create-book-response.png`).

**Apport de SOAP :** la structure XML et le WSDL définissent explicitement les opérations, leurs paramètres et leurs types. Les clients doivent suivre ce contrat, ce qui facilite l'interopérabilité mais exige davantage de configuration et produit des messages plus longs que les appels internes du monolithe.

# API REST et conteneurisation

L'API `rest/main.py` utilise FastAPI. Les modèles Pydantic `BookIn` et `BookUpdate` contrôlent les champs reçus : ID strictement positif à la création, texte non vide, stock et tarif positifs ou nuls. `rest/db.py` assure l'accès SQLAlchemy à PostgreSQL.

| Méthode | Route | Comportement dans le code |
|---|---|---|
| GET | `/books` | Retourne la liste des livres, triée par ID. |
| POST | `/books` | Crée un livre ; réponse 201, doublon 409. |
| PUT | `/books/{id}` | Met à jour un livre ; absent 404. |
| DELETE | `/books/{id}` | Supprime un livre ; réponse 204, absent 404. |

FastAPI génère la documentation interactive à `http://localhost:8000/docs` et le schéma OpenAPI. Le `Dockerfile` construit l'API sur Python 3.11 ; `rest/docker-compose.yml` relie le conteneur de l'API au réseau de la base PostgreSQL.

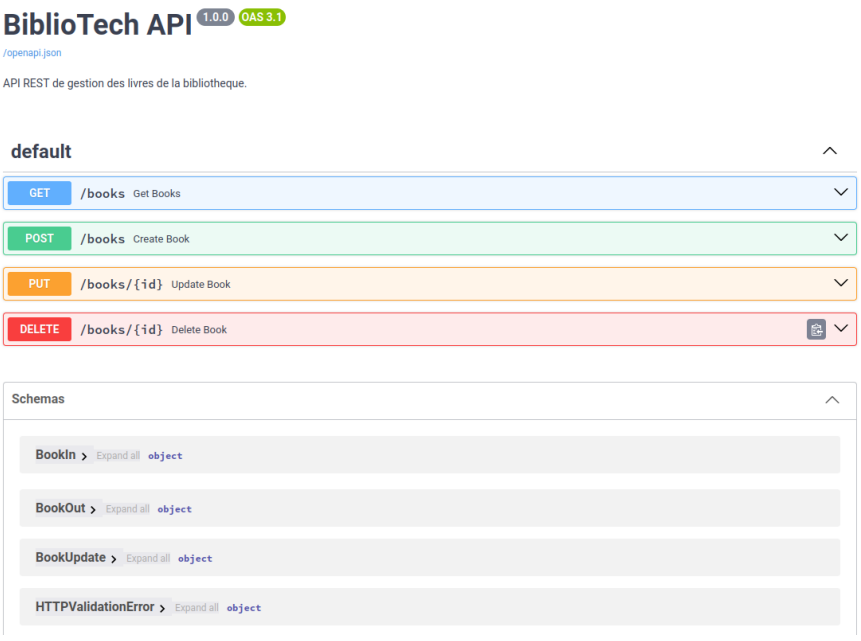



Figure 3 — Routes OpenAPI conservées dans `rest/assets/api-endpoints.png`.

Une autre capture, `rest/assets/get-books-response.png`, montre une réponse JSON à `GET /books`.

# Comparaison et bilan

| Critère | SOAP | REST |
|---|---|---|
| Format | Enveloppes XML | Requêtes HTTP et réponses JSON |
| Contrat | WSDL explicite | Schéma OpenAPI généré par FastAPI |
| Lisibilité | Messages plus détaillés et verbeux | Corps JSON et routes plus directs à lire |
| Utilisation | Client conforme au WSDL requis | Outils HTTP courants et interface `/docs` |
| Erreurs | SOAP Fault pour validation et livre absent | Codes HTTP 404, 409, 422 selon le cas |

### Correspondance avec l'énoncé

**Partie 1** Le dépôt contient le script unique, le modèle, les opérations CRUD, la validation et une capture de l'interface ; le README répond à la question sur les limites du monolithe.

**Partie 2** Les quatre opérations SOAP, le WSDL, la connexion SQLAlchemy, les validations et les captures Postman sont présents ; le README explique le contrat SOAP.

**Partie 3** Les quatre routes REST, les modèles Pydantic, la documentation OpenAPI, le Dockerfile et Compose sont présents ; le README compare SOAP et REST.

### Conclusion

Le parcours du lab montre le passage d'un programme local qui concentre tous les traitements à deux interfaces de service réutilisables. SOAP formalise fortement les échanges grâce au WSDL. REST expose les mêmes besoins métier à travers des routes HTTP, des modèles de validation et une documentation automatiquement générée.

> **Portée de la vérification :** les sources Python, les liens de documentation et le XML du WSDL ont été contrôlés statiquement. Le lancement des conteneurs et des tests Pytest n'a pas été refait dans cet environnement, où Docker et Pytest sont absents. Les résultats illustrés proviennent des captures déjà présentes dans le dépôt.